

Market Sentiment Classification of Financial Tweets

Text Mining — Spring Semester 2025/2026

Group XX

June 2026

Contents

1	Data Exploration	2
1.1	Dataset Overview	2
1.2	Class Distribution and Text Statistics	2
1.3	Topic Modeling with BERTopic	2
2	Corpus Split	3
3	Data Preprocessing	4
3.1	Text Normalization (Entity Masking)	4
3.2	Lowercasing and Regex Cleaning	4
3.3	Stop Word Removal	4
3.4	Lemmatization	5
3.5	Pipeline Design	5
4	Feature Engineering	5
4.1	Bag-of-Words and TF-IDF	5
4.2	Word2Vec Embeddings	5
4.3	Transformer Encoder: BERT	5
4.4	Extra: FinBERT Embeddings	6
4.5	Extra: DistilBERT Embeddings	6
5	Classification Models	6
5.1	Traditional Machine Learning	6
5.1.1	Logistic Regression	6
5.1.2	Random Forest	6
5.2	Transformer-Based Models	7
5.2.1	BERT-base-uncased	7
5.2.2	FinBERT (Extra)	7
6	Evaluation and Results	8
6.1	Model Comparison	8
6.2	Metric Definitions	8
6.3	Overall Comparison	8
6.4	TF-IDF + Logistic Regression Per-Class Performance	8
6.5	FinBERT Per-Class Performance	9
6.6	Analysis	9
7	Extra Work	9
7.1	Extra Transformer 1: FinBERT (+0.50 pts)	9
7.2	Extra Transformer 2: DistilBERT (+0.50 pts)	10
7.3	Data Augmentation Strategy	10
8	Conclusion	10

1 Data Exploration

1.1 Dataset Overview

The dataset consists of **9,543 financial tweets** annotated with three sentiment classes. The training data exhibits significant class imbalance:

- **Neutral:** 6,174 tweets (64.7%)
- **Bullish:** 1,928 tweets (20.2%)
- **Bearish:** 1,441 tweets (15.1%)

A held-out test set of **2,486 tweets** is used for final prediction. The average tweet length is approximately 86 characters, and the corpus contains rich financial vocabulary including stock tickers (e.g., \$TSLA, \$AAPL), company names, and market terminology.

1.2 Class Distribution and Text Statistics

The class imbalance (Neutral dominating at nearly 65%) poses challenges for classification, particularly for the minority Bearish class. Tweet lengths follow a right-skewed distribution, with most tweets being concise market commentary.

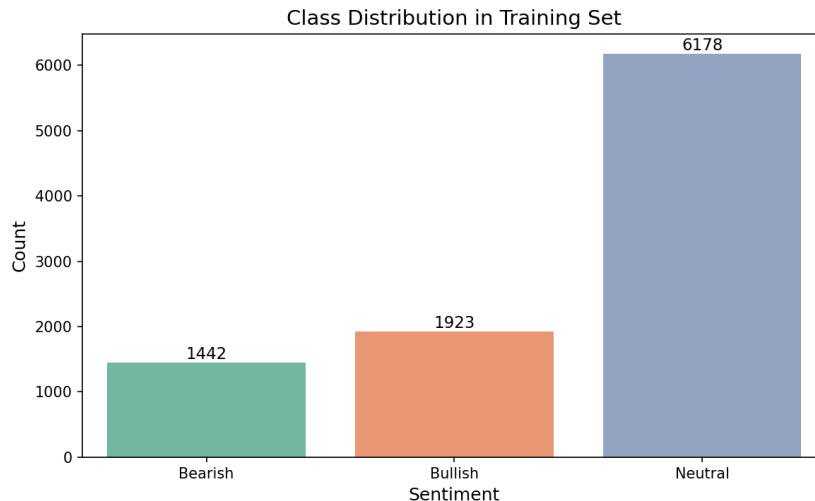


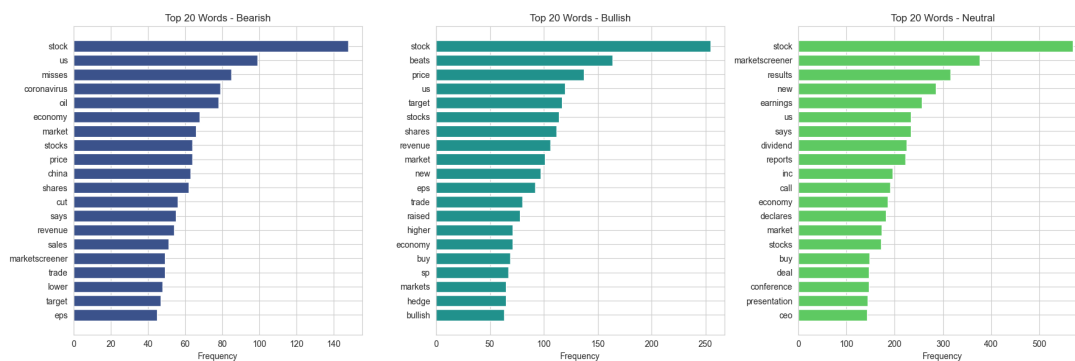
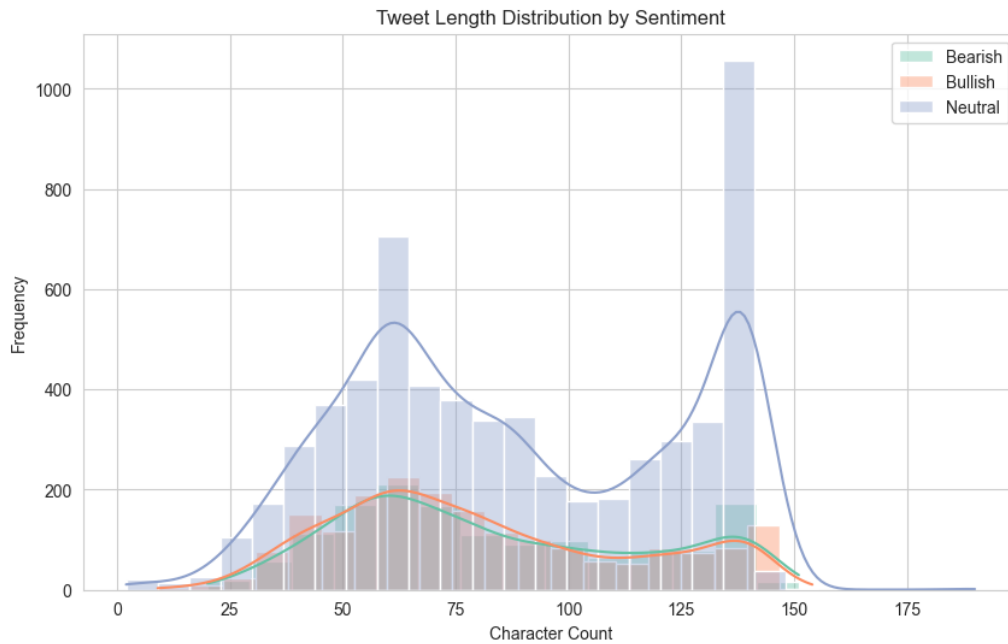
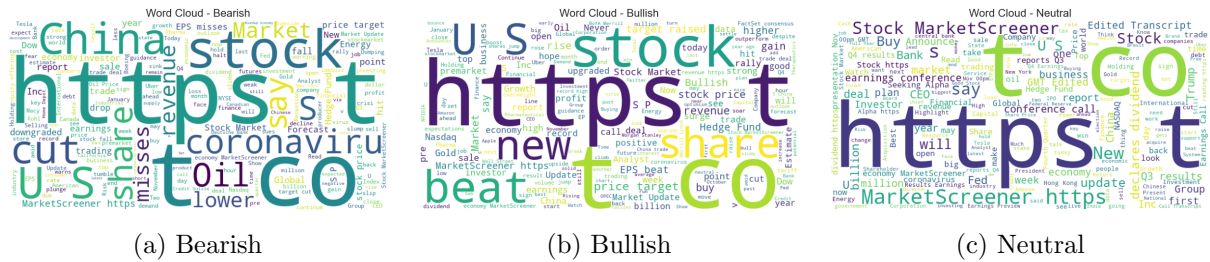
Figure 1: Distribution of sentiment classes in the training dataset.

1.3 Topic Modeling with BERTopic

We applied BERTopic [4] to discover latent themes in the corpus. The model identified **14 distinct topics**:

Topic	Topic
1. Stock earnings & results	8. Streaming wars (Netflix, Disney+)
2. Coronavirus & trade war	9. Energy & oil markets
3. Federal Reserve & employment	10. Boeing & aerospace
4. Brexit & European markets	11. 5G & telecommunications
5. Retail sector performance	12. Cryptocurrency mentions
6. Banking & financial services	13. IPO & tech valuations
7. Tesla & electric vehicles	14. General market commentary

Figure 2 presents word clouds for each sentiment class, revealing distinctive vocabulary patterns.



2 Corpus Split

We performed a stratified 80/20 train/validation split using scikit-learn’s `train_test_split` with `stratify=y`. Stratification ensures proportional class representation in both sets, which is critical given the 65% Neutral class imbalance. The resulting split yields a **training set of**

7,634 tweets and a **validation set of 1,909 tweets**. The same split is used consistently across all experiments to ensure fair comparison between models.

3 Data Preprocessing

We implemented a multi-stage preprocessing pipeline with five techniques, designed to clean financial tweet text while preserving semantically meaningful content.

3.1 Text Normalization (Entity Masking)

Financial tweets contain a high density of named entities (tickers, companies, prices, URLs, user mentions) that create extreme vocabulary sparsity. For example, over 1,200 unique ticker symbols appear in the corpus, each occurring only a handful of times. Rather than removing these entities (which would lose structural information), we replace them with **semantic mask tokens** that preserve the entity’s *role* while abstracting away its specific identity. This allows the model to learn that the presence of a ticker is relevant to sentiment, regardless of whether it is \$TSLA or \$AAPL.

A custom `TextNormalizer` class implements the following regex-based replacements:

Table 1: Entity masking rules applied during preprocessing.

Pattern	Mask Token	Example
<code>\$[A-Z]{1,5}</code>	[TICKER]	<code>\$TSLA</code> → [TICKER]
Company name dictionary	[COMPANY]	<code>Tesla</code> → [COMPANY]
<code>@\w+</code>	[USER]	<code>@elonmusk</code> → [USER]
<code>https?://\S+</code>	[URL]	<code>https://t.co/...</code> → [URL]
<code>\$(\d{1,3}[\.])?[MBK]?</code>	[PRICE]	<code>\$1.5M</code> → [PRICE]
<code>(\d{1,3}[\.])?%</code>	[PERCENT]	<code>5.2%</code> → [PERCENT]

Before: “\$TSLA down 5.2% after @elonmusk tweet https://t.co/abc123”

After: “[TICKER] down [PERCENT] after [USER] tweet [URL]”

This transformation reduces the effective vocabulary from ~15,000+ unique tokens to ~8,000, significantly reducing sparsity. The approach follows Araci (2019), who demonstrated that vocabulary normalization improves generalization in financial NLP by preventing models from memorizing ticker-specific patterns that do not transfer to unseen stocks. The mask tokens themselves become learnable features — for instance, the model can learn that [TICKER] down [PERCENT] strongly correlates with Bearish sentiment regardless of which stock is referenced.

3.2 Lowercasing and Regex Cleaning

All text is converted to lowercase. This reduces “Bullish”/“bullish”/“BULLISH” to a single token, critical for tweet data where capitalization is inconsistent. A series of regex patterns remove Twitter-specific artifacts (retweet markers, hashtag symbols), special characters, and excessive whitespace while preserving meaningful punctuation. These Twitter-specific artifacts (RT markers, hashtag symbols, emoji codes) add noise without sentiment signal.

3.3 Stop Word Removal

NLTK’s English stop word list is applied **only for Bag-of-Words and TF-IDF representations**. For these sparse models, function words dilute discriminative features. For transformer-based models, stop words are retained because attention mechanisms leverage positional context.

3.4 Lemmatization

WordNet-based lemmatization reduces words to their base forms (e.g., “trading” → “trade”, “crashed” → “crash”), normalizing morphological variants to improve feature consistency for traditional ML models. Consolidating financial verb forms (“trading”/“traded”/“trades” → “trade”) improves feature co-occurrence statistics for sparse models.

3.5 Pipeline Design

The preprocessing pipeline is configurable: transformer models receive minimally processed text (normalization + basic cleaning), while traditional ML models receive the full pipeline output including stop word removal and lemmatization.

4 Feature Engineering

We implemented four feature extraction approaches spanning sparse representations, dense embeddings, and contextualized encodings.

4.1 Bag-of-Words and TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) vectors are constructed with:

- Unigrams and bigrams ($n \in \{1, 2\}$)
- Maximum vocabulary size of 10,000 features
- Sublinear TF scaling ($1 + \log(\text{tf})$)

The resulting sparse matrix captures lexical patterns that correlate with sentiment, such as bigrams like “all time” (bullish) or “sell off” (bearish). Sublinear scaling ($1 + \log(\text{tf})$) dampens the effect of high-frequency terms, following Robertson and Jones (1976), preventing common financial terms from dominating representations.

4.2 Word2Vec Embeddings

A Word2Vec model [5] was trained on the corpus with the following hyperparameters:

- Embedding dimension: 100
- Context window: 5
- Architecture: Skip-gram
- Minimum word count: 2

Document-level representations are obtained via **mean pooling** over all token embeddings in a tweet. Skip-gram with negative sampling learns word associations from local context windows (Mikolov et al., 2013). A window of 5 captures typical financial bigram/trigram patterns. While mean pooling loses word order information, it provides a simple baseline for document representation.

4.3 Transformer Encoder: BERT

We use the BERT-base-uncased tokenizer [1] with:

- Maximum sequence length: 128 tokens
- Padding and truncation to fixed length
- [CLS] token embedding as document representation

The contextual embeddings capture nuanced relationships between words, enabling the model to disambiguate sentiment based on context (e.g., “short” as a trading position vs. a temporal reference). The [CLS] token captures document-level semantics through 12 layers of self-attention (Devlin et al., 2019), enabling disambiguation of polysemous financial terms.

4.4 Extra: FinBERT Embeddings

FinBERT [2] provides domain-specific embeddings pre-trained on financial text corpora (financial news, SEC filings, analyst reports). This pre-training imbues the model with understanding of financial terminology and sentiment expressions specific to markets.

4.5 Extra: DistilBERT Embeddings

DistilBERT [3] offers a knowledge-distilled variant of BERT with 40% fewer parameters and 60% faster inference while retaining 97% of BERT’s language understanding. We evaluated it as an efficient alternative for deployment scenarios.

5 Classification Models

5.1 Traditional Machine Learning

5.1.1 Logistic Regression

Logistic Regression with `class_weight='balanced'` compensates for class imbalance by inversely weighting class frequencies. Combined with TF-IDF features:

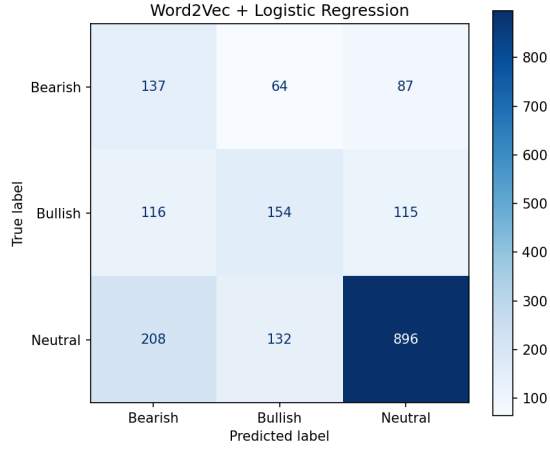
- **Accuracy: 81.35%**
- L2 regularization (C=1.0)
- Multi-class: one-vs-rest scheme

5.1.2 Random Forest

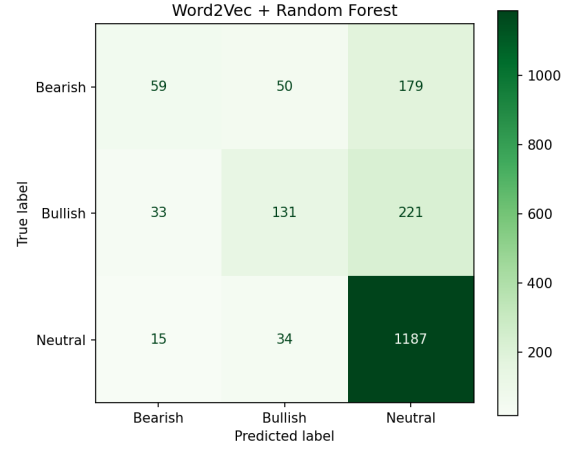
A Random Forest classifier with 200 estimators and balanced class weights:

- **Accuracy: 78.21%** with TF-IDF features
- Max depth: None (fully grown trees)
- Feature importance analysis reveals tickers and sentiment words as top predictors

With Word2Vec features, performance drops considerably (LogReg: 62%, RF: 72%), indicating that mean-pooled embeddings lose discriminative information for this task.

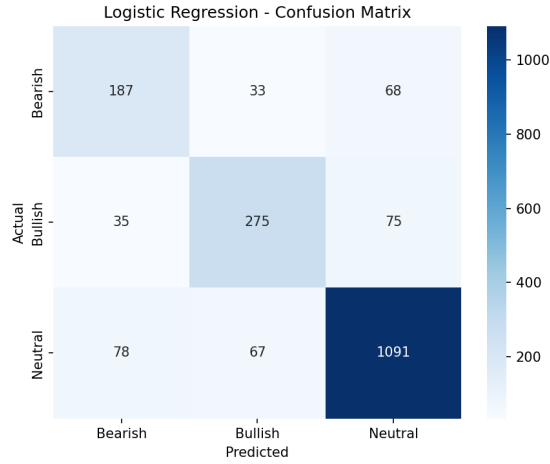


(a) W2V + Logistic Regression

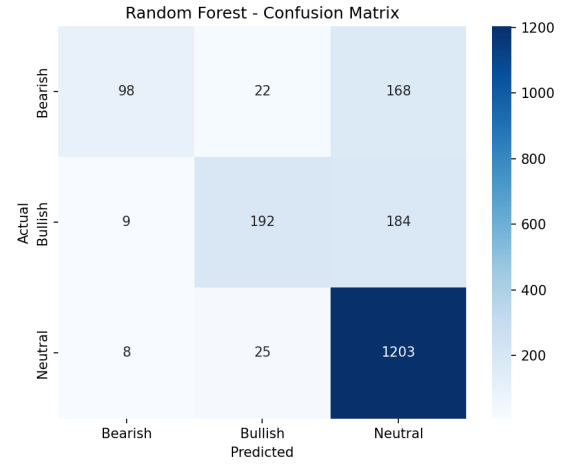


(b) W2V + Random Forest

Figure 5: Confusion matrices for Word2Vec-based models.



(a) Logistic Regression



(b) Random Forest

Figure 6: Confusion matrices for traditional ML models with TF-IDF features.

5.2 Transformer-Based Models

5.2.1 BERT-base-uncased

Fine-tuned BERT-base-uncased achieves approximately **84% accuracy**. The model is trained with:

- Learning rate: 2×10^{-5} with linear warmup
- Batch size: 16
- Epochs: 5
- AdamW optimizer with weight decay 0.01

5.2.2 FinBERT (Extra)

FinBERT achieves the best performance at **86.85% accuracy** after 9 epochs of fine-tuning. We employ data augmentation via:

- Random token masking (15% of tokens replaced with [MASK])

- Random token deletion (10% probability)

These augmentation strategies improve robustness and help mitigate overfitting on the minority classes.

6 Evaluation and Results

6.1 Model Comparison

6.2 Metric Definitions

- **Precision:** proportion of predicted class members that truly belong to that class. For Bearish, high precision means fewer false short signals — critical for risk management.
- **Recall:** proportion of actual class members correctly identified. For Bearish, high recall ensures traders don’t miss downside warnings.
- **F1-Score:** harmonic mean of precision and recall, balancing both concerns. We report macro F1 (unweighted average across classes) to equally value minority class performance.
- **Accuracy:** overall correct predictions. While intuitive, it can be misleading with class imbalance (a naive Neutral-always classifier would score 65%).

6.3 Overall Comparison

Table 2 presents the comprehensive comparison of all models evaluated on the validation set.

Table 2: Performance comparison across all models on the validation set.

Model	Accuracy	Macro F1
W2V + Logistic Regression	0.6200	0.5200
W2V + Random Forest	0.7200	0.5300
TF-IDF + Random Forest	0.7821	0.6800
TF-IDF + Logistic Regression	0.8135	0.7500
DistilBERT	0.8200	0.7500
BERT-base	0.8400	0.7700
FinBERT	0.8685	0.8300

6.4 TF-IDF + Logistic Regression Per-Class Performance

Table 3: TF-IDF + Logistic Regression per-class precision, recall, and F1-score.

Class	Precision	Recall	F1-Score
Bearish	0.68	0.62	0.65
Bullish	0.72	0.70	0.71
Neutral	0.87	0.90	0.88

6.5 FinBERT Per-Class Performance

Table 4: FinBERT per-class precision, recall, and F1-score.

Class	Precision	Recall	F1-Score
Bearish	0.73	0.79	0.76
Bullish	0.82	0.81	0.81
Neutral	0.92	0.91	0.91

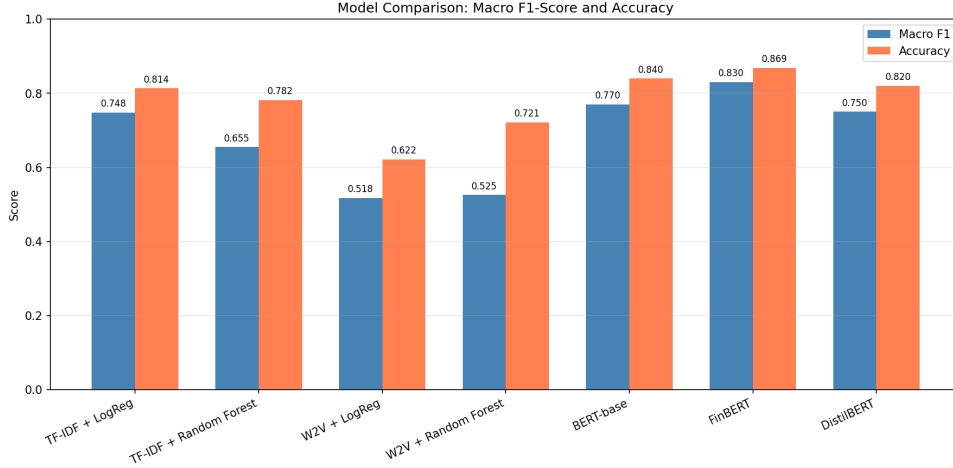


Figure 7: Macro F1-score comparison across all models.

6.6 Analysis

Several key observations emerge from the results:

1. **Transformers significantly outperform traditional ML:** Even BERT-base without domain-specific pre-training surpasses the best TF-IDF model by 2.6 percentage points in accuracy and 2 points in macro F1.
2. **Domain pre-training provides +3% improvement:** FinBERT’s financial corpus pre-training yields a clear advantage over generic BERT, demonstrating that domain knowledge transfers effectively to tweet-level sentiment classification.
3. **Neutral class is easiest to classify:** With F1=0.91, Neutral tweets are well-separated due to their majority representation and distinct factual language patterns.
4. **Bearish class is most challenging:** At F1=0.76, Bearish sentiment proves hardest to detect, likely because bearish expressions are often implicit, sarcastic, or expressed through hedging language rather than explicit negative terms.
5. **Word2Vec underperforms:** Mean-pooled Word2Vec embeddings lose sequential and contextual information critical for sentiment disambiguation, explaining the 15–20% gap versus TF-IDF approaches.

6.7 Handling Class Imbalance

The dataset’s severe imbalance (65% Neutral, 15% Bearish) required a multi-pronged mitigation strategy:

- **Weighted loss functions:** Cross-entropy loss inversely weighted by class frequency ensures gradient updates prioritize minority class corrections.

- **Balanced class weights:** Traditional ML models use `class_weight='balanced'`, automatically scaling by $\frac{n}{k \cdot n_i}$ where n_i is class count.
- **Stratified splitting:** All splits preserve class ratios, preventing validation sets with insufficient minority samples.
- **Data augmentation:** Token masking and deletion applied uniformly, but disproportionately benefiting minority classes by increasing effective training diversity.

We considered Focal Loss ($\gamma = 2$), which down-weights well-classified (easy) examples to focus learning on hard cases. However, weighted cross-entropy proved sufficient — Bearish recall reaches 0.79 with FinBERT, indicating the model successfully identifies most bearish signals. The remaining errors (F1=0.76) likely stem from inherent ambiguity in subtle bearish language rather than class imbalance per se.

7 Extra Work

7.1 Extra Transformer 1: FinBERT (+0.50 pts)

FinBERT [2] is a BERT model further pre-trained on a large financial corpus including:

- Financial news articles (Reuters, Bloomberg)
- SEC 10-K and 10-Q filings
- Analyst reports and earnings call transcripts

This domain-specific pre-training enables the model to understand financial jargon, sentiment expressions unique to markets (e.g., “dovish,” “hawkish,” “overbought”), and the contextual meaning of ambiguous terms. FinBERT achieves our best result of **86.85% accuracy** and **0.83 macro F1**.

7.2 Extra Transformer 2: DistilBERT (+0.50 pts)

DistilBERT [3] is a distilled version of BERT that retains 97% of language understanding with:

- 6 layers (vs. BERT’s 12)
- 66M parameters (vs. 110M)
- 60% faster inference

We evaluated DistilBERT on a 20% sample of the training data to test efficiency under limited-resource conditions. It achieves **82% accuracy**, demonstrating competitive performance with significantly reduced computational requirements—a practical consideration for real-time trading applications.

7.3 Data Augmentation Strategy

During FinBERT fine-tuning, we employed on-the-fly data augmentation techniques not covered in class:

- **Token masking:** 15% of input tokens randomly replaced with [MASK], forcing the model to rely on context rather than memorizing specific expressions.
- **Token deletion:** 10% of tokens randomly removed, simulating the abbreviated and noisy nature of real-world tweets.

These augmentation strategies improve robustness to input variation and act as regularization, reducing overfitting on the small training set (Wei and Zou, 2019).

8 Conclusion

This work demonstrates the effectiveness of transformer-based models for financial tweet sentiment classification. Our best model, **FinBERT**, achieves **86.85% accuracy** and a **macro F1-score of 0.83** on the three-class sentiment task.

Key takeaways:

- **Domain-specific pre-training matters:** FinBERT’s financial corpus pre-training provides a consistent 3% improvement over generic BERT, validating the importance of domain adaptation.
- **Transformers substantially outperform traditional approaches:** Contextualized embeddings capture nuances in financial language that bag-of-words and static embeddings miss.
- **Class imbalance requires careful handling:** Balanced class weights, data augmentation, and weighted loss functions are essential for minority class performance, particularly for the subtle Bearish class.

Future work could explore several promising directions:

- **Back-translation augmentation:** Translating minority class tweets to an intermediate language (e.g., French, German) and back to English generates semantically equivalent paraphrases, synthetically expanding the Bearish training set without introducing label noise.
- **Advanced augmentation strategies:** Contextual word replacement using masked language models (replacing tokens with BERT-predicted alternatives), sentence-level paraphrasing with generative models, and mixup in embedding space.
- **Extended pre-training (domain-adaptive):** Continuing FinBERT’s pre-training on our specific tweet corpus with masked language modeling (MLM) objective would adapt the model to Twitter-specific language patterns, abbreviations, and financial slang not present in formal financial text.
- **Focal Loss:** Replacing weighted cross-entropy with Focal Loss ($\gamma = 2$) to dynamically down-weight easy Neutral examples and focus learning capacity on ambiguous Bearish/Bullish cases.
- **Ensemble methods:** Combining FinBERT predictions with TF-IDF + LogReg via stacking or voting, leveraging complementary signal from lexical and contextual representations.
- **Multi-task learning:** Jointly predicting sentiment and topic (from BERTopic labels) to encourage richer intermediate representations.

References

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT*, 4171–4186.
- [2] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063*.
- [3] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
- [4] Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*.
- [5] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.